Proof-Oriented Validation of Natural-Language Intents for Autonomous Networks

Anonymous Authors

Abstract—Natural language (NL) intents will be increasingly the control mechanism of 5G/6G autonomous telecom networks.

Example intent: "Provide an ultra-reliable low-latency 5G service for telemedicine and critical care operations at Mayo Clinic on an ongoing basis. Support up to 80 critical devices and 200 auxiliary endpoints. Maintain end-to-end latency below 10 ms …"

This shift creates a safety and reliability problem: natural-language or weakly structured intents are attractive at the management plane, but they can be ambiguous, underspecified, and may not validate against additional provider-specific operational constraints.

This paper presents a prototype, high-assurance, intent-admission shell for autonomous networks built around the TM Forum TMF921 Intent Management API, which serves as a "front door" for NL intents and as such imposes the minimal semantic requirements through an OWL ontology. However any specific telecom provider implementation of TMF921 will have additional requirements that are not covered by the ontology.

This paper shows how natural-language intents can be trusted through proof-relevant guarantees provided by formal verification systems based on dependently typed languages. First (offline) the OWL ontology is converted into the corresponding dependent types of the F* language. These serve as propositions against which incoming intents may be checked. Then (online) received natural-language intents are translated by a Large Language Model (LLM) to F* code that is validated (type-checked) against the ontology-derived dependent types using the F* proof checker. Only valid intents are admitted.

Further, since TMF921 structure-imposition is necessarily generic, we demonstrate provider-specific intent validation via further refinements of the ontology types.

It is observed that valid intents can be processed with high reliability after mild prompt tuning. On a 300-trial evaluation over ten semantically correct expressions, GPT-5.4 achieved 100% first-attempt success in generating F* artifacts that passed the proof-oriented checks.

Index Terms—autonomous networks, large language models, formal verification, dependent types

I. Introduction

Autonomous networks are a particularly demanding instance of a broader systems problem: humans and software agents can easily generate natural language requests, but fluent output is not the same as safe or correct output. Similar boundary problems already appear in configuration validation [1], natural-language firewall configuration [2], and production multi-agent network management [3]. Recent agent-security work argues that systems should apply defense-in-depth and complete mediation rather than treat model outputs as trustworthy by default [4]. This concern is particularly acute for network control workflows where accepted requests can eventually influence provisioning, routing, or remediation.

At this boundary, two distinct sources of nondeterminism must be controlled. The first is the natural-language request itself – the same meaning can be conveyed in different surface forms. The second is the LLM-generated artifact used to interpret that request, whose output may vary across runs even when the intended meaning is unchanged. Proof-checked output validation counters both by accepting only interpretations that can be turned into the required machine-checkable artifact.

This paper describes a prototype shell around the TM Forum [5] TMF921 Intent Management API [6] that addresses this admission problem. TMF921 allows for intent submission in natural language, structured JSON, or weakly structured JSON formats. Here the focus is on the natural-language path.

First, offline, the TM Forum intent ontology is projected into F* [7], [8] dependent types that capture the standards-aligned semantics of the API [9], [10]. Then, online, incoming requests at the TMF921 boundary are normalized and, for the natural-language path, translated by an LLM into F* artifacts that must type-check against that ontology-derived layer before admission. Around that checked path, the system emits a canonical intent intermediate representation (IR), normalized JSON-LD [11], generated F* modules and other items for traceability and diagnostics.

Our argument is not that formal methods alone solve autonomous-network safety, nor that natural-language interfaces should be avoided. Instead, we argue for a narrow and practical claim: the boundary between LLM generation and provider execution is the right place to combine standards-aligned API handling, semantic normalization, and machine-checked admissibility tests. Autonomous networking is the motivating domain here, but the pattern applies more generally wherever model-generated artifacts must be highly trusted before they can allocate resources, change policy, or trigger control-plane actions.

A. Contributions

This paper makes two main contributions:

1) It frames dependent-type, proof-oriented validation as an admission-layer pattern for high-trust LLM outputs and instantiates that pattern as a TMF921 compliant API [12], [13].

```
//declare a function type that takes a nat and returns a nat
type T1 = nat -> nat

//this function satisfies type T1 (as would many others)
let addOne: T1 =  fun x -> x + 1
```

Listing 1. Simple function type in F*.

```
//first define a dependent type for even numbers
//as a constraint on nat
type Even (n:nat) = _:unit { n % 2 = 0 }

//then define a dependent function type that
//requires Even input and output
type T2 = (x: nat{Even x}) -> nat{Even result}

//this function satisfies type T2 because it takes an
//even number and produces an even number
let addTwo: T2 = fun x -> x + 2

//this function does NOT satisfy type T2 because it
//takes an even number but produces an odd number
let addOneError: T2 = fun x -> x + 1
```

Listing 2. More complex dependent function type in F*.

2) It separates general intent validity from provider-specific admissibility, allowing failures such as under-specification, infeasible requests, and policy violations to be localized. This is achieved by successively refining the dependent types derived from the TM Forum ontology into provider-specific types. This system allows different providers to use the same standards-aligned core while layering on their own constraints.

II. Dependent Types and Proof-Oriented Validation

Via the Curry–Howard correspondence [14], propositions can be represented as types and proofs as programs; in our setting, we encode the required intent obligations as F* types. Then, the LLM-generated artifact (the program) is a candidate proof of those types. Successful type checking serves as machine-checked evidence of compliance. Practical proof-oriented systems such as Lean [15] and F* [8] make this style of checked construction usable for real systems.

To illustrate, consider a simple function type T1 given in Listing 1, whose inputs and outputs are natural numbers.

Since T1 is inhabited via the function addOne, it means that addOne is a proof of T1. So under Curry–Howard the type T1 can be read as a proposition that is proven by the program addOne.

Now let's consider a slightly more complex function type that accepts and returns even numbers. We can express this using dependent types given in Listing 2. Here, Even n is a dependent type that represents the proposition that a natural number is even. Subsequently, the function type T2 requires that inputs and outputs should be of type Even n.

The type Even n is a dependent type because it depends on the value n which is supplied from where the type is used. Here n in Even n is used in the constraint n % 2 = 0 that is part of the type declaration. In fact any arbitrary constraint can be used as part of a type declaration. In summary, in a dependent type system, types can depend on values and other functions. However, this simple explanation does not do justice to the full expressiveness of dependent types, but it gives a flavor of how they can encode constraints. In principle, dependently typed programming languages can encode any mathematical theorem expressible in the underlying formal language [16].

It suffices to understand that a complex set of constraints can be encoded as dependent types. Even state machines can be encoded as types. The key point is that system (or business) constraints/rules can be modelled as types. Then, the validation (of code that uses these types) is automatic via the proof-oriented type checkers. Non-dependently typed languages – with simpler type checkers – cannot express such constraints as types.

Dependent types are highly compositional, letting complex constraints be constructed from simpler ones. This compositionality is essential for modeling real-world systems, where constraints often interact in subtle ways and evolve over time.

Once the systems/business rules are modeled as types then LLM-generated programs can be validated against them, thus eliminating the source of non-determinism inherent in LLM-generated outputs.

III. The Unreasonable Effectiveness of Dependent Types

Experiments in this prototype show that current frontier models can generate correct F* artifacts reliably on the first attempt for this narrow benchmark once the prompt makes the required surface syntax explicit, which was far less plausible in the late-2024 GPT-4o/o1 period. Table I compares GPT-5.4 and GPT-4o using only benchmarks that appear under the same name in official OpenAI sources. The overlap is limited, but the shared results still indicate a substantial recent increase in reasoning and agentic reliability. The same trend is reflected in the emergence and rise of dedicated agentic coding products such as Anthropic's Claude Code [17], OpenAI's Codex [18], and GitHub Copilot [19].

A plausible contributor to this improvement is the broader shift toward reinforcement learning with verifiable rewards (RLVR) [20] on tasks with objective checkers, including mathematics, code, and formal reasoning. Public evidence for proprietary frontier models remains limited. However, Google DeepMind states that the advanced Gemini Deep Think model used for International Mathematical Olympiad (IMO)-level reasoning [21] was additionally trained with novel reinforcement learning techniques that leverage multi-step reasoning, problem-solving, and theorem-proving data [22]. In the open literature, Tulu 3 explicitly identifies RLVR as a post-training method [23], and recent Lean-based work shows how proof assistants can serve as verifier-grounded reward

TABLE I
Official OpenAI-reported results on benchmarks with exact name overlap between GPT-4o and GPT-5.4 [31]–[33].

Benchmark	GPT-4o	GPT-5.4
GPQA Diamond	46.0%	92.8%
BrowseComp	1.9%	82.7%

Note: The BrowseComp entry uses the browsing-enabled GPT-4o result reported by OpenAI, since GPT-5.4 was evaluated on BrowseComp with ChatGPT search enabled.

oracles during theorem-proving training [24]. While these sources do not establish that GPT-5.4 itself was post-trained on Lean traces, they do show that verifiable-reward training over formal reasoning tasks is now a credible part of the broader frontier-model toolkit.

The likely involvement of dependently-typed languages in post-training of frontier LLMs (either directly or indirectly) is an indication towards their satisfactory performance on dependent-type code generation.

While the dependently typed Lean language [16], [25] is favored for LLM training, F* is more suited for automatic validation of LLM-generated artifacts because it supports a higher degree of proof automation. F* relies heavily on the Z3 satisfiability modulo theories (SMT) solver to automate proofs [8], [26], [27]. Lean, on the other hand, was built as an interactive proof assistant and is oriented toward human-assisted proof construction using tactics [25], [28].

The F* type checker statically analyzes source code to prove correctness. As such it can handle much more complexity than what its taxed with when validating TMF921 intents. However these simple, declarative intents actually drive complex control code to manage dynamic networks. There is no reason why LLM code generation cannot be taken further to generate the actual control code from the parsed intents. Type-based checks and balances can still apply but at a more granular level allowing for composable constraints enabling flexible intent expressions to possibly handle previously unanticipated situations.

Type-checked F* code can be converted to other languages for more convenient execution. One such target language is F#, a general-purpose programming language [29]. The TMF921 Web API and shell processing is implemented in F# which internally invokes F*. Both F* and F# stem from the same lineage - ML [30]. F# can be considered a non-dependently typed version of F*. F# thus can be the target of LLM-generated but F* - checked, more complex control code.

IV. Processing Overview

The prototype shell implementation has two distinct stages: offline and online. Offline, a selected subset of the TM Forum intent ontology is projected into F* modules that define the standards-aligned baseline used by the checker [7]–[10], [13]. Online, the TMF921 expression field is extracted, translated into F* artifacts when needed, and admitted only if both standards-aligned and provider-specific predicates hold. Additional artifacts are also emitted for traceability and diagnostics.

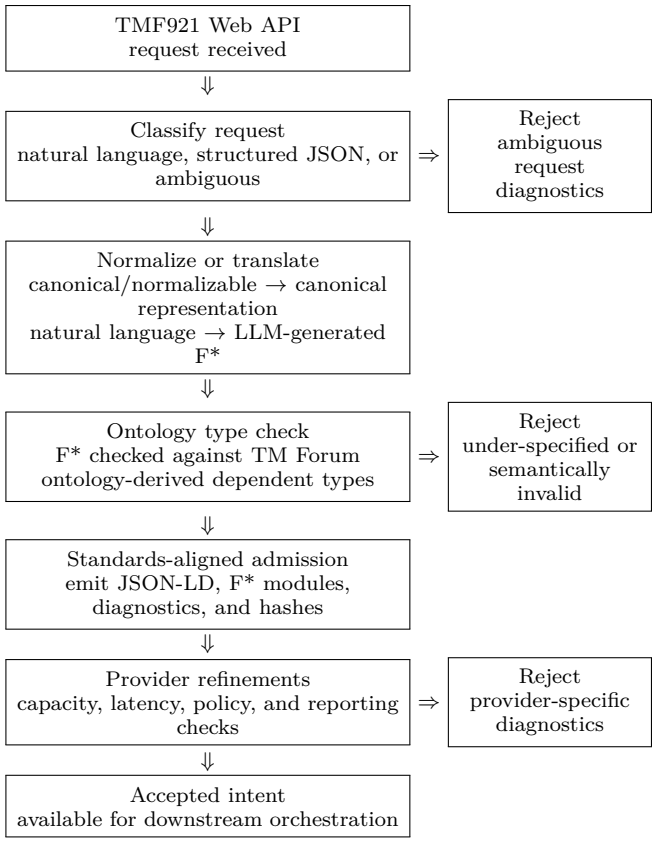

Fig. 1. Online TMF921 intent-admission pipeline. The standards-aligned gate checks whether the request is actionable under the ontology-derived dependent types; provider refinements then check local admissibility constraints.

Figure 1 summarizes this online path and shows where rejection diagnostics are produced.

V. Example Ontology Fragment and Refinement

To illustrate the projection and refinement process, consider a live-broadcast scenario family with the following sample request:

"Provide a premium 5G broadcast service for the live event at Detroit Stadium on April 25, 2026 from 18:00 to 22:00 EST. Support up to 200 production devices. Keep uplink latency under 20 ms. Send hourly compliance updates and immediate alerts if service quality degrades. Do not impact emergency-service traffic."

Figure 2 shows the compact ontology fragment used to interpret this request. The fragment is intentionally small: an intent has a target (one or more services), a delivery window, measurable quantities, a reporting expectation, and a safety or security posture. For the live-broadcast scenario, these fields are enough to distinguish a vague request from an actionable intent without exposing the reader to the full ontology.

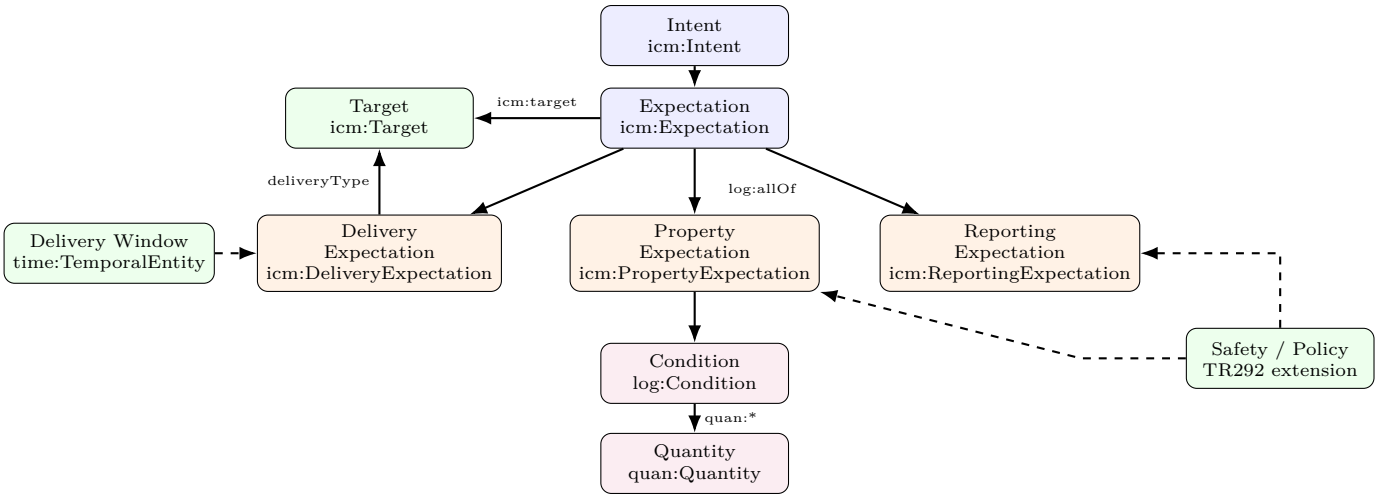

Fig. 2. Compact ontology fragment used by the case study. An intent is interpreted as a conjunction of delivery, property, and reporting expectations over a target. Property expectations reduce to checked conditions over quantities; temporal context and policy posture constrain admissibility.

Listing 3 is an elided but still representative view of the projection of the common-core into F* types: the request must name a target and service, provide a complete window and reporting expectation, and attach positive measurable quantities. The quantity_checked_intent type is a refinement of the measurable_intent type (plus other constraints). The LLM-generated raw_tm_intent, from the input expression, is further refined by quantity_checked_intent. When full F* code is assembled, the type checker will reject any intent that fails to meet the common-core requirements.

Provider checks are then layered on top of the common core. In the running live-broadcast example, Detroit Stadium resolves to the LiveBroadcastGold profile, which admits at most 250 production devices and requires a latency bound no tighter than 20 ms. The accepted scenario requests 200 devices, 20 ms uplink latency, hourly reporting, immediate degradation alerts, and no emergency-traffic preemption.

This witness structure localizes failures. A request missing a target, window, or quantity cannot construct the common-core witness; a request with excessive device count, an impossible latency claim, or public-safety preemption can pass the ontology-derived gate and still fail the provider witness. This is the separation between general intent validity and local provider admissibility.

The definition of provider-specific types as refinements of the core types is an example of compositionality of dependent types.

```
let structural_ok (i:raw_tm_intent) : Type0 =
  let e = intent_expression_of i in
  let m = metric_requirements_of i in
  target_present e.projection_target /\
  service_present e.projection_service_class /\
  delivery_window_present e.projection_delivery_window /\
  reporting_expectation_present e.projection_reporting /\
  required_metrics_present
    (requires_auxiliary_metric i)
    m

let quantities_ok (i:raw_tm_intent) : Type0 =
  all_quantities_positive
    (requires_auxiliary_metric i)
    (primary_device_quantity_of i)
    (auxiliary_endpoint_quantity_of i)
    (max_latency_quantity_of i)
    (reporting_interval_quantity_of i)

let provider_ok (p:profile) (i:raw_tm_intent) : Type0 =
  window_ok i /\
  profile_matches p i /\
  capacity_ok p i /\
  latency_ok p i /\
  policy_ok i

type structurally_actionable_intent =
  i:raw_tm_intent { structural_ok i }

type quantity_wellformed_intent =
  i:structurally_actionable_intent { quantities_ok i }

type provider_admissible_intent (p:profile) =
  i:quantity_wellformed_intent { provider_ok p i }
}
```

Listing 3. Simplified common-core F* excerpt derived from the ontology fragment.

VI. LLM Performance and Observations

We evaluated the natural-language path on ten synthetic but semantically correct requests: five live-broadcast expressions and five critical-service expressions, each chosen from scenarios that should be admitted by the type checker. Each expression was run 30 times with GPT-5.4, for a total of 300 end-to-end trials per prompt version.

There are two levels of retry allowed where invalid generated code is re-submitted to the LLM for correction along with failure diagnostics.

With mild prompt tuning, the LLM was able to generate correct F* code on the first attempt for the intents in the evaluation set. With the initial prompt version, the first-

```
type profile =
  | LiveBroadcastSilver
  | LiveBroadcastGold
  | UnsupportedProfile

let resolve_profile (i:raw_tm_intent) : profile =
  match i.scenario_family,
        (target_descriptor_of i).descriptor_name with
  | BroadcastFamily, Some "Detroit Stadium" ->
      LiveBroadcastGold
  | BroadcastFamily, Some "Metro Arena" ->
      LiveBroadcastSilver
  | _ ->
      UnsupportedProfile

let profile_matches (p:profile) (i:raw_tm_intent) : Type0 =
  resolve_profile i == p /\ p <> UnsupportedProfile

type window_checked_intent =
  i:quantity_wellformed_intent {
    window_ok i
  }

type profiled_intent (p:profile) =
  i:window_checked_intent {
    profile_matches p i
  }

type provider_checked_intent (p:profile) =
  i:profiled_intent p {
    capacity_ok p i /\
    latency_ok p i /\
    policy_ok i
  }
```

Listing 4. Simplified provider refinement excerpt for live broadcast admission.

TABLE II
End-to-end GPT-5.4 results on 10 accepted synthetic expressions, repeated 30 times each.

Model / prompt	First attempt	Within 1 retry	Within 2 retries
GPT-5.4 / v2	300/300		

attempt success rate was 270/300.

Early failures showed that the model was semantically close to the correct artifact, but often wrong in two pieces of F* syntax: event_month was sometimes emitted as Some 4 instead of Some "April", and intent_name was sometimes wrapped as Some "..." even though the field requires a bare quoted string. We therefore revised the prompt to include an exact raw_tm_intent template, explicit field-formatting rules, and concrete "wrong/right" examples for those recurrent syntax errors. The final prompt revision used for the reported results is included in Appendix A.

Using the final prompt version, the rerun achieved 300/300 successes on the first attempt, with no first-attempt parser issues observed in the saved trial records. The first-attempt rate was therefore 100%, with a 95% Wilson interval of approximately 98.74–100%.

VII. Limitations and Future Work

The current evidence is architectural rather than broad empirical. The natural-language handling is narrow, the provider model is simplified, and the evaluation covers limited scenario families. The next steps are to broaden the scenario set, connect admission checks to dynamic state via control code generation to understand the power and limits of LLM-generated but proof-validated code.

VIII. Related Work

Prior work already points toward validated intermediate artifacts for LLM-assisted systems. Ciri studies output checking for configuration validation [1], while Taghiyev and Aslanbayli use natural language only to produce a typed IR for firewall rules [2]. Meaning-Typed Programming similarly argues for semantically structured intermediate representations in LLM-integrated applications [34]. Our work follows the same direction, but emphasizes proof-oriented admissibility at the execution boundary.

On the safety side, Zhang et al. argue that LLM agents should adopt classic security principles such as defense-in-depth and complete mediation [4]. In networking, the Confucius framework shows that multi-agent systems already rely on validation layers and existing tooling to reduce regressions [3]. Our contribution is complementary: we focus on the earlier boundary where candidate requests first become eligible for authoritative handling.

IX. Conclusion

In conclusion, this work positions dependent types (encoded in F*) as a practical mechanism for managing the inherent non-determinism of agentic, LLM-driven systems at critical control boundaries such as the TMF921 intent interface. Rather than treating variability in natural language and model outputs as a liability, the approach channels it through proof-oriented validation aligned with the TMF921 standard, where only artifacts that construct valid proofs of intent constraints are admitted. This reframes dependent types not merely as a safety feature, but as a modeling discipline: business rules, operational constraints, and evolving policies are encoded directly as composable types and refinements derived from the TMF921 ontology. Such compositionality enables clean layering—from standards-aligned semantics to provider-specific constraints—making systems easier to manage, audit, and evolve while preserving high assurance.

Appendix A
Prompts

The following is the prompt revision used for the final GPT-5.4 evaluation runs. The system message contains the stable normalization instructions; each trial instantiates the user-message template with one natural-language request and, on repair attempts, the prior validation diagnostics.

```
You are a semantic normalization component for TMF921 intent admission.

Task:
- Convert natural-language telecom-management intent text into a restricted F*
      intent module.
- Return only structured data that conforms to the requested schema.

Output contract:
- If the request is specific enough to map into the supported admission subset, return
      status = "parsed" and moduleText containing exactly one F* module.
- If the request is too vague or missing core semantics, return status = "
      clarification_required" with issues and omit moduleText.
```

```
Allowed F* shape:
- The module must open ‘TmForumTr292CommonCore‘.
- It must declare exactly one ‘let candidate_intent : raw_tm_intent = { ... }‘ record.
- Prefer the module name ‘GeneratedIntent‘.
- Use only these record fields:
  intent_name
  scenario_family
  target_name
  target_kind
  service_class
  event_month
  event_day
  event_year
  start_hour
  end_hour
  timezone
  primary_device_count
  auxiliary_endpoint_count
  max_latency_ms
  reporting_interval_minutes
  immediate_degradation_alerts
  safety_policy_declared
  preserve_emergency_traffic
  request_public_safety_preemption

Allowed enum values:
- scenario_family: BroadcastFamily | CriticalServiceFamily
- target_kind: Some VenueTarget | Some FacilityTarget | None

Exact field formatting rules:
- ‘intent_name‘ is a required string, so write ‘"..."‘, never ‘Some "..."‘.
- ‘target_name‘, ‘service_class‘, ‘timezone‘, and ‘event_month‘ are optional strings,
      so write either ‘None‘ or ‘Some "..."‘.
- ‘event_month‘ must be a month name string such as ‘Some "April"‘, never ‘Some
      4‘.
- ‘event_day‘, ‘event_year‘, ‘start_hour‘, ‘end_hour‘, ‘primary_device_count‘, ‘
      auxiliary_endpoint_count‘, ‘max_latency_ms‘, and ‘
      reporting_interval_minutes‘ are optional naturals, so write either ‘None‘ or ‘
      Some 25‘.
- ‘target_kind‘ is an optional enum, so write ‘Some VenueTarget‘, ‘Some
      FacilityTarget‘, or ‘None‘.
- Boolean fields must be bare ‘true‘ or ‘false‘.

Canonical module template:
‘‘‘fstar
module GeneratedIntent

open TmForumTr292CommonCore

let candidate_intent : raw_tm_intent =
  { intent_name = "LiveBroadcastIntent";
    scenario_family = BroadcastFamily;
    target_name = Some "Detroit Stadium";
    target_kind = Some VenueTarget;
    service_class = Some "premium-5g-broadcast";
    event_month = Some "April";
    event_day = Some 25;
    event_year = Some 2026;
    start_hour = Some 18;
    end_hour = Some 22;
    timezone = Some "America/Detroit";
    primary_device_count = Some 200;
    auxiliary_endpoint_count = None;
    max_latency_ms = Some 20;
    reporting_interval_minutes = Some 60;
    immediate_degradation_alerts = true;
    safety_policy_declared = true;
    preserve_emergency_traffic = true;
    request_public_safety_preemption = false }
‘‘‘

Common syntax mistakes to avoid:
- Wrong: ‘intent_name = Some "LiveBroadcastIntent"‘
- Right: ‘intent_name = "LiveBroadcastIntent"‘
- Wrong: ‘event_month = Some 4‘
- Right: ‘event_month = Some "April"‘

Rules:
- Extract semantics from the user's text without inventing targets, dates, times,
      quantities, or policy facts.
- Preserve explicit bad-but-stated semantics, such as reversed time windows.
- Do not perform provider-admission reasoning.
- For broadcast requests, set ‘intent_name = "LiveBroadcastIntent"‘.
- For critical-service requests, set ‘intent_name = "CriticalServiceIntent"‘.
- For broadcast requests, use service_class = Some "premium-5g-broadcast" when
      the text clearly indicates premium 5G broadcast service.
- For critical-service requests, use service_class = Some "ultra-reliable-5g-clinical"
      when the text clearly indicates telemedicine, clinical, or critical-care service.
- Set safety_policy_declared = true only if the text explicitly states either protected
      -traffic preservation or public-safety preemption.
- For BroadcastFamily, auxiliary_endpoint_count should be None unless the text
      explicitly includes a second endpoint quantity.
- For CriticalServiceFamily, auxiliary_endpoint_count should be Some <nat> only
      when the text explicitly includes auxiliary endpoints.
- If the text is too vague to identify a target, service class, and measurable
      expectations without guessing, return clarification_required.

Few-shot guidance:
1. Accepted broadcast:
Input: "Provide a premium 5G broadcast service for the live event at Detroit
      Stadium on April 25, 2026 from 18:00 to 22:00 America/Detroit. Support up
      to 200 production devices. Keep uplink latency under 20 ms. Send hourly
      compliance updates and immediate alerts if service quality degrades. Do not
      impact emergency-service traffic."
Behavior: return parsed with scenario_family = BroadcastFamily, target_name =
      Some "Detroit Stadium", target_kind = Some VenueTarget,
      primary_device_count = Some 200, reporting_interval_minutes = Some 60,
      immediate_degradation_alerts = true, safety_policy_declared = true,
      preserve_emergency_traffic = true, request_public_safety_preemption =
      false.

2. Accepted critical service:
Input: "Provide an ultra-reliable 5G clinical service for telemedicine and critical care
      operations at Mayo Clinic on April 25, 2026 from 08:00 to 20:00 America/
      Detroit. Support up to 80 critical devices and 200 auxiliary endpoints.
      Maintain end-to-end latency below 10 ms. Send compliance updates every 5
      minutes and immediate alerts if service quality degrades. Do not impact
      emergency-service traffic."
Behavior: return parsed with scenario_family = CriticalServiceFamily, target_name
      = Some "Mayo Clinic", target_kind = Some FacilityTarget,
      primary_device_count = Some 80, auxiliary_endpoint_count = Some 200,
      max_latency_ms = Some 10, reporting_interval_minutes = Some 5.

3. Reversed window:
Input: "Provide premium 5G broadcast service at Detroit Stadium on April 25, 2026
      from 22:00 to 18:00 America/Detroit with latency under 20 ms and hourly
      reports."
Behavior: return parsed and preserve start_hour = Some 22 and end_hour = Some
      18 without correcting them.

4. Vague request:
Input: "Make the event network really good and fast for the broadcast."
Behavior: return clarification_required with issues describing the missing target and
      measurable expectations.

Current prompt version: 2026-04-23.fstar-intent.v2
```

Listing 5. System prompt for version 2026-04-23.fstar-intent.v2.

```
Normalize the following natural-language input into a restricted F* intent module.

Natural-language intent:
"""<natural-language intent text>"""

<optional repair section>
Server validation failures from the previous attempt:
- <diagnostic code>: <diagnostic message> Details: <diagnostic details>

Repair only those failures while preserving the user's stated semantics.

Reminder:
- Return only the structured response.
- When parsed, moduleText must contain exactly one F* module using the allowed
      record shape.
- Follow the canonical field formatting exactly, especially for ‘intent_name‘ and ‘
      event_month‘.
- Do not emit provider witness code or additional helper functions.
- Do not invent unstated facts.
```

Listing 6. User-message template paired with the system prompt.

References

[1] X. Lian, Y. Chen, R. Cheng, J. Huang, P. Thakkar, M. Zhang, and T. Xu, “Configuration Validation with Large Language Models,” arXiv:2310.09690, 2023. [Online]. Available: https://arxiv.org/abs/2310.09690

[2] F. Taghiyev and A. Aslanbayli, “Natural Language Interface for Firewall Configuration,” arXiv:2512.10789, 2025. [Online]. Available: https://arxiv.org/abs/2512.10789

[3] Z. Wang, S. Lin, G. Yan, S. Ghorbani, M. Yu, J. Zhou, N. Hu, L. Baruah, S. Peters, S. Kamath, J. Yang, and Y. Zhang, “Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework,” in Proceedings of ACM SIGCOMM 2025, 2025. [Online]. Available: https://doi.org/10.1145/3718958.3750537

[4] K. Zhang, Z. Su, P.-Y. Chen, E. Bertino, X. Zhang, and N. Li, “LLM Agents Should Employ Security Principles,” arXiv:2505.24019, 2025. [Online]. Available: https://arxiv.org/abs/2505.24019

[5] TM Forum, “TM Forum Home,” 2026, accessed April 22, 2026. [Online]. Available: https://www.tmforum.org/

[6] ——, “Intent Management API TMF921-v5.0,” 2026, accessed April 22, 2026. [Online]. Available: https://www.tmforum.org/open-digital-architecture/open-apis/intent-management-api-TMF921/v5.0

[7] The F* Project, “F*: A Proof-Oriented Programming Language,” 2026, accessed April 22, 2026. [Online]. Available: https://fstar-lang.org/

[8] N. Swamy, C. Hritcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoue, and S. Zanella-Beguelin, “Dependent types and multi-monadic effects in F*,” in Proceedings of POPL 2016, 2016. [Online]. Available: https://fstar-lang.org/papers/mumon/

[9] TM Forum, “TR290A Intent Common Model Part 1: Intent Expression v3.6.0,” 2024, technical report.

[10] ——, “TR292D Quantity Ontology v3.6.0,” 2024, technical report.

[11] G. Kellogg, P.-A. Champin, and D. Longley, "JSON-LD 1.1: A JSON-based Serialization for Linked Data," W3C Recommendation, 2020. [Online]. Available: https://www.w3.org/TR/json-ld11/
[12] TM Forum, "TMF921 Intent Management API v5.0.0 Specification," 2024, technical specification.
[13] ——, "TR292 TM Forum Intent Ontology (TIO) v3.6.0," 2024, technical report.
[14] P. Wadler, "Propositions as types," Communications of the ACM, vol. 58, no. 12, pp. 75–84, 2015. [Online]. Available: https://doi.org/10.1145/2699407
[15] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer, "The lean theorem prover (system description)," in Automated Deduction – CADE-25, 2015, pp. 378–388. [Online]. Available: https://doi.org/10.1007/978-3-319-21401-6_26
[16] J. Avigad and P. Massot, "Mathematics in Lean," 2025, version 4.19.0; accessed April 25, 2026. [Online]. Available: https://leanprover-community.github.io/mathematics_in_lean/
[17] Anthropic, "Claude Code Overview," 2026, accessed April 23, 2026. [Online]. Available: https://code.claude.com/docs/en/overview
[18] OpenAI, "Codex | AI Coding Partner from OpenAI," 2026, accessed April 23, 2026. [Online]. Available: https://openai.com/codex/
[19] GitHub, "GitHub Copilot Features," 2026, gitHub Docs; accessed April 23, 2026. [Online]. Available: https://docs.github.com/en/copilot/get-started/features
[20] Y. Mroueh, "Reinforcement Learning with Verifiable Rewards: GRPO's Effective Loss, Dynamics, and Success Amplification," arXiv:2503.06639, 2025. [Online]. Available: https://arxiv.org/abs/2503.06639
[21] International Mathematical Olympiad, "International Mathematical Olympiad," 2026, official website; accessed April 23, 2026. [Online]. Available: https://math.imo-official.org/
[22] T. Luong and E. Lockhart, "Advanced Version of Gemini with Deep Think Officially Achieves Gold-Medal Standard at the International Mathematical Olympiad," 2025, google DeepMind blog, published July 21, 2025; accessed April 23, 2026. [Online]. Available: https://deepmind.google/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/
[23] N. Lambert, J. Morrison, V. Pyatkin, S. Huang, H. Ivison, F. Brahman, L. J. V. Miranda, A. Liu, N. Dziri, S. Lyu, Y. Gu, S. Malik, V. Graf, J. D. Hwang, J. Yang, R. L. Bras, Ø. Tafjord, C. Wilhelm, L. Soldaini, N. A. Smith, Y. Wang, P. Dasigi, and H. Hajishirzi, "Tulu 3: Pushing Frontiers in Open Language Model Post-Training," arXiv:2411.15124, 2024. [Online]. Available: https://arxiv.org/abs/2411.15124
[24] M. Kim and S.-Y. Yun, "Process-Verified Reinforcement Learning for Theorem Proving via Lean," in NeurIPS 2025 Workshop MATH-AI, 2025, poster paper. [Online]. Available: https://openreview.net/forum?id=sKvVHgmRP2
[25] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in Automated Deduction – CADE 28, 2021, pp. 625–635. [Online]. Available: https://doi.org/10.1007/978-3-030-79876-5_37
[26] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in Tools and Algorithms for the Construction and Analysis of Systems, 2008, pp. 337–340. [Online]. Available: https://doi.org/10.1007/978-3-540-78800-3_24
[27] C. Barrett and C. Tinelli, "Satisfiability Modulo Theories," in Handbook of Satisfiability, 2nd ed. IOS Press, 2021. [Online]. Available: https://theory.stanford.edu/~barrett/pubs/BT18.pdf
[28] Lean FRO, "The Lean Language Reference: Tactic Proofs," 2026, accessed April 25, 2026. [Online]. Available: https://lean-lang.org/doc/reference/latest/Tactic-Proofs/
[29] Microsoft, "What is F#?" 2026, accessed April 22, 2026. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/fsharp/what-is-fsharp
[30] R. Milner, "A Theory of Type Polymorphism in Programming," Journal of Computer and System Sciences, vol. 17, no. 3, pp. 348–375, 1978. [Online]. Available: https://doi.org/10.1016/0022-0000(78)90014-4
[31] OpenAI, "Introducing GPT-5.4," 2026, published March 5, 2026; accessed April 23, 2026. [Online]. Available: https://openai.com/index/introducing-gpt-5-4/
[32] ——, "Introducing GPT-4.1 in the API," 2025, published April 14, 2025; accessed April 23, 2026. [Online]. Available: https://openai.com/index/gpt-4-1/
[33] ——, "BrowseComp: a Benchmark for Browsing Agents," 2025, published April 10, 2025; accessed April 23, 2026. [Online]. Available: https://openai.com/index/browsecomp/
[34] J. Mars, Y. Kang, J. L. Dantanarayana, K. Sivasothynathan, C. Clarke, B. Li, K. Flautner, and L. Tang, "Meaning-Typed Programming: Language-level Abstractions and Runtime for GenAI Applications," arXiv:2405.08965, 2024. [Online]. Available: https://arxiv.org/abs/2405.08965